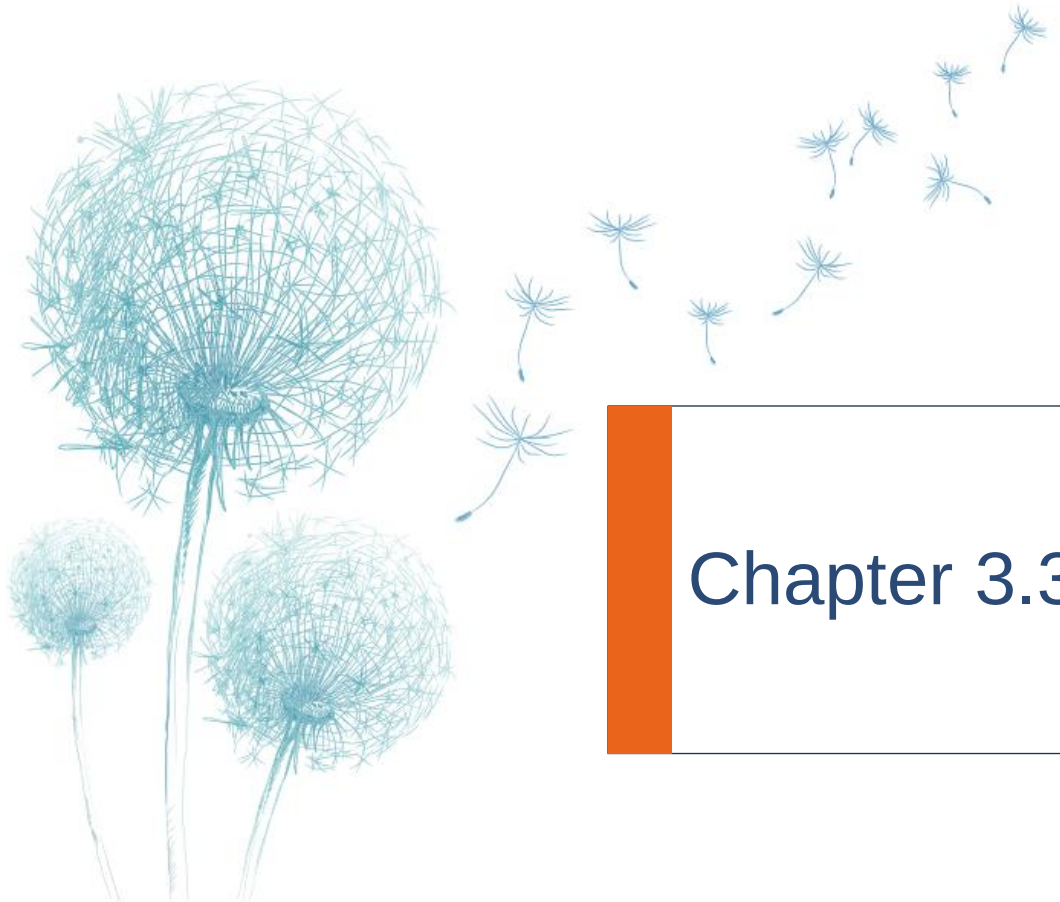
DS4023（2024 Fall）
Machine Learning
Department of Computer Science
BNU-HKBU United International College



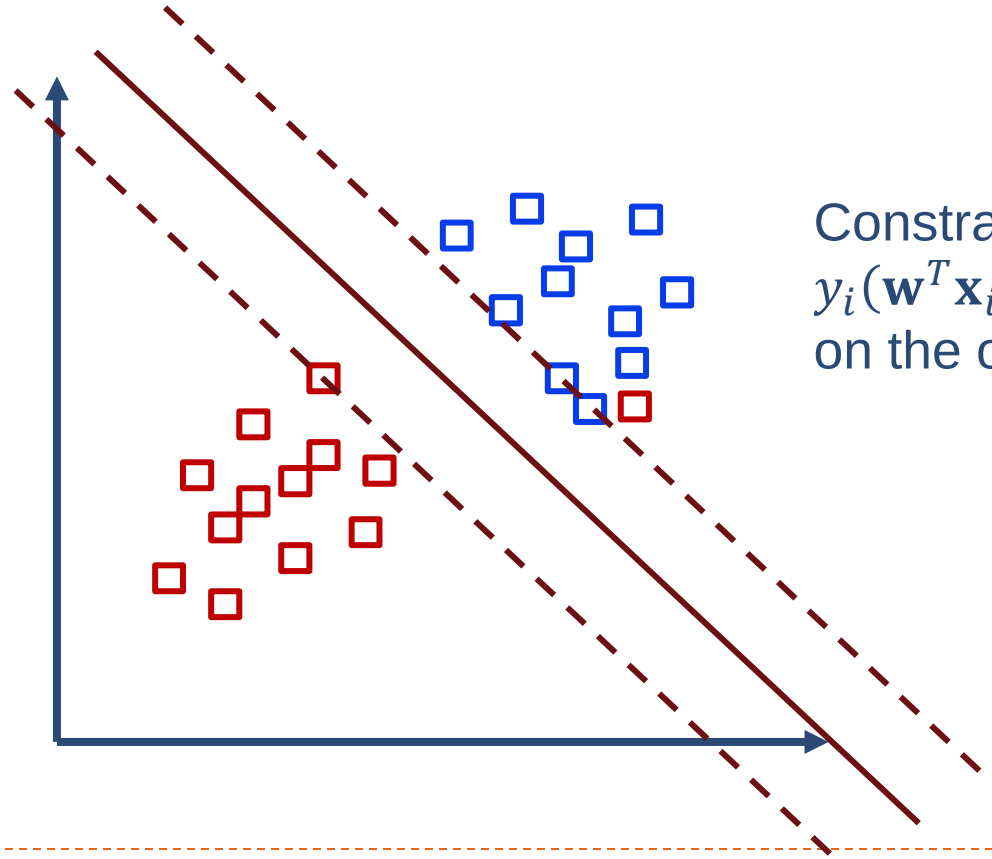
Chapter 3.3: Support Vector Machine II

Outline

- Support vector machine model
- Constrained optimization
- Soft margin
- Kernel methods

Non-linearly separable case

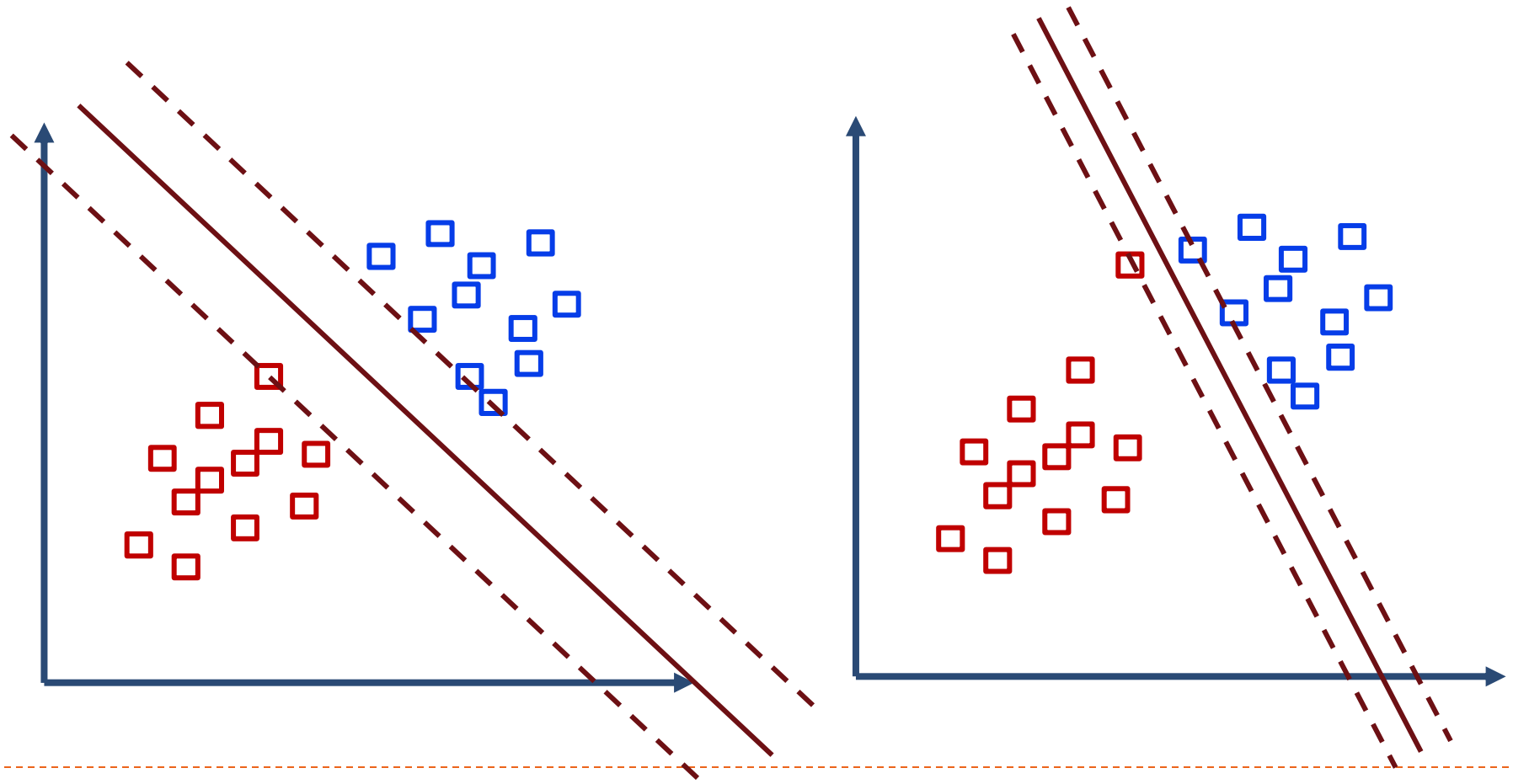
- What if training data not linearly separable due to noise or outliers?



Constraint
$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$ is violated on the outlier/noise

Linearly separable with small margin

- Suppose we add an outlier which results in a classifier with a much smaller margin.



Soft Margin

- To make the SVM model work for non-linearly separable datasets as well as be less sensitive to outliers
  - allow error for some samples
- Allow some sample violated the constraint:
  - $y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 < 0$
  - soft margin, not hard margin
- Target: find a hyperplane that
  - maximize margin
  - minimize number of errors

Soft Margin

- Introduce positive slack variables ξ_i, constraints:
  - $$\begin{cases} \mathbf{w}^T\mathbf{x}_i + b \geq 1 - \xi_i & y_i = 1 \\ \mathbf{w}^T\mathbf{x}_i + b \leq -(1 - \xi_i) & y_i = -1 \\ \xi_i \geq 0 & \forall i \end{cases}$$
- New constraints:
  - $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$
- Penalize ξ_i in the objective function
  - for an error to occur, the corresponding ξ_i must exceed unity

Optimization problem with soft margin

- The optimization problem using soft margin:

$$\min_{\mathbf{w},\mathrm{b}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$$
$$s.t.\ \ y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i,\ \ i = 1,2,\dots,m$$
$$\xi_i \geq 0, \qquad i = 1,2,\dots,m$$

- Examples are now permitted to have margin less than 1 and if an example whose margin is $1 - \xi_i$, we would pay a cost of the objective function being increased by $C\xi_i$
  - Hinge loss: $l_{hinge}(z) = \max(0,1 - z)$
  - Another view of SVM:

$$\min_{\mathbf{w},\mathrm{b}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max(0,1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$$

Optimization problem with soft margin

- The parameter C controls the relative weighting between the twin goals of making
  - the $\|\mathbf{w}\|^2$ small
  - ensuring that most examples have margin greater than 1
- For large C value: the optimization will choose a smaller-margin hyperplane if that hyperplane does a better job of getting all the training points classified correctly.
- For small C value: the optimizer to look for a larger-margin separating hyperplane, even if that hyperplane misclassifies more points.

Optimization problem with soft margin (optional)

- Similarly, it is a constrained (convex) quadratic optimization problem. We construct the Lagrangian for this problem:
- $\mathcal{L}(w, b, \xi, \alpha, \mu) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\xi_i +$

$$\sum_{i=1}^{m}\alpha_i[1 - \xi_i - y_i(\mathbf{w}^T\mathbf{x}_i + b)] - \sum_{i=1}^{m}\mu_i\,\xi_i$$

  - where $\alpha_i \geq 0, \mu_i \geq 0$ are lagrange multipliers
- Find the dual problem do by setting the derivatives of $\mathcal{L}$ with respect to w, b and ξ to zero:

$$\frac{\partial}{\partial_w}\mathcal{L}(w, b, \xi, \alpha, \mu) = w - \sum_{i=1}^{m}\alpha_i y_i\,\mathbf{x}_i = 0$$

$$\frac{\partial}{\partial_b}\mathcal{L}(w, b, \xi, \alpha, \mu) = \sum_{i=1}^{m} -\alpha_i y_i = 0$$

$$\frac{\partial}{\partial_{\xi_i}}\mathcal{L}(w, b, \xi, \alpha, \mu) = C - \alpha_i - \mu_i = 0$$

Optimization problem with soft margin (optional)

- Dual problem:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i{}^T \mathbf{x}_j$$

$$s.t. \quad C \geq \alpha_i \geq 0, \qquad i = 1,2,\dots,m$$

$$\sum_{i=1}^{m} \alpha_i y_i = 0$$

- There exists $w^*, b^*, \xi^*, \alpha^*, \mu^*$, so that w^*, b^*, ξ^* is the solution to the primal problem and α^*, μ^* are solution to the dual problem and $p^* = d^* = \mathcal{L}(w^*, b^*, \xi^*, \alpha^*, \mu^*)$
  - Moreover, $w^*, b^*, \xi^*, \alpha^*, \mu^*$ satisfy the KKT conditions

Optimization problem with soft margin (optional)

- Similar with hard margin svm model, besides the partial derivative of w^*, b^*, ξ^* be zero, the KKT conditions are :

$$1 - \xi_i^* - y_i\left(\mathbf{w}^{*T}\mathbf{x}_i + b^*\right) \leq 0, \xi_i^* \geq 0\ , \forall i \qquad (1)$$

$$\alpha_i^* \geq 0, \mu_i^* \geq 0, \forall i \qquad (2)$$

$$\alpha_i^*(1 - \xi_i - y_i\left(\mathbf{w}^{*T}\mathbf{x}_i + b^*\right)) = 0, \forall i \qquad (3)$$

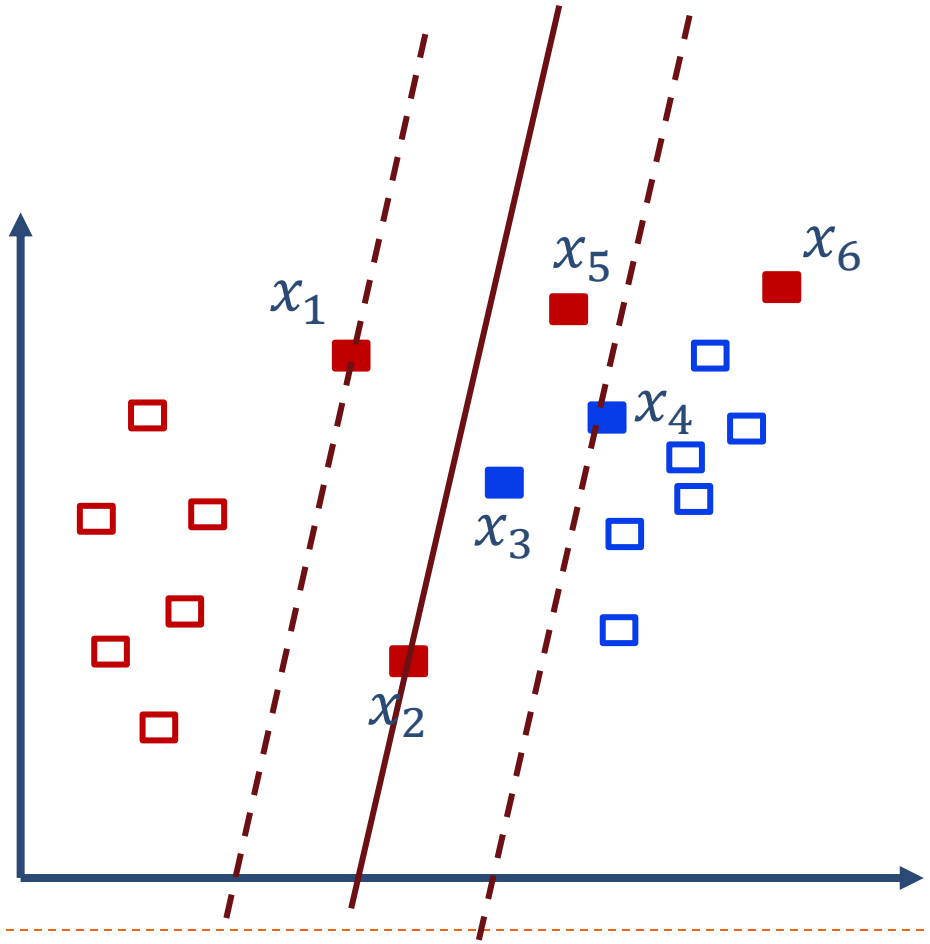
$$\mu_i^*\xi_i^* = 0, \forall i \qquad (4)$$

Soft Margin SVM

- For any sample $(\mathbf{x}_i, y_i)$
  - $\alpha_i^* = 0, \mu_i = \mathrm{C}$, therefore $\xi_i = 0$, sample lies beyond the margin.
  - $0 < \alpha_i^* < C, \mu_i > 0$, therefore $\xi_i = 0$ and $1 - \xi_i - y_i(\mathbf{w}^T\mathbf{x}_i + b) = 0$ sample lies on the maximum margin (support vectors)
  - $\alpha_i^* = C, \mu_i = 0$, if $\xi_i \leq 1$, sample lies inside the maximum margin; $\xi_i > 1$, misclassified (error)
- For a solution $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_m^*)^T$ for the dual problem, if exist α_j^* that $0 < \alpha_j^* < C$, then the solution for primal problem
  - $w^* = \sum_{i=1}^{m} \alpha_i^* y_i \mathbf{x}_i$
  - $b^* = y_j - \sum_{i=1}^{m} \alpha_i^* y_i \mathbf{x}_i{}^T \mathbf{x}_j$

Soft Margin SVM

- What are the values or range of ξ_i for the data samples in the given example?

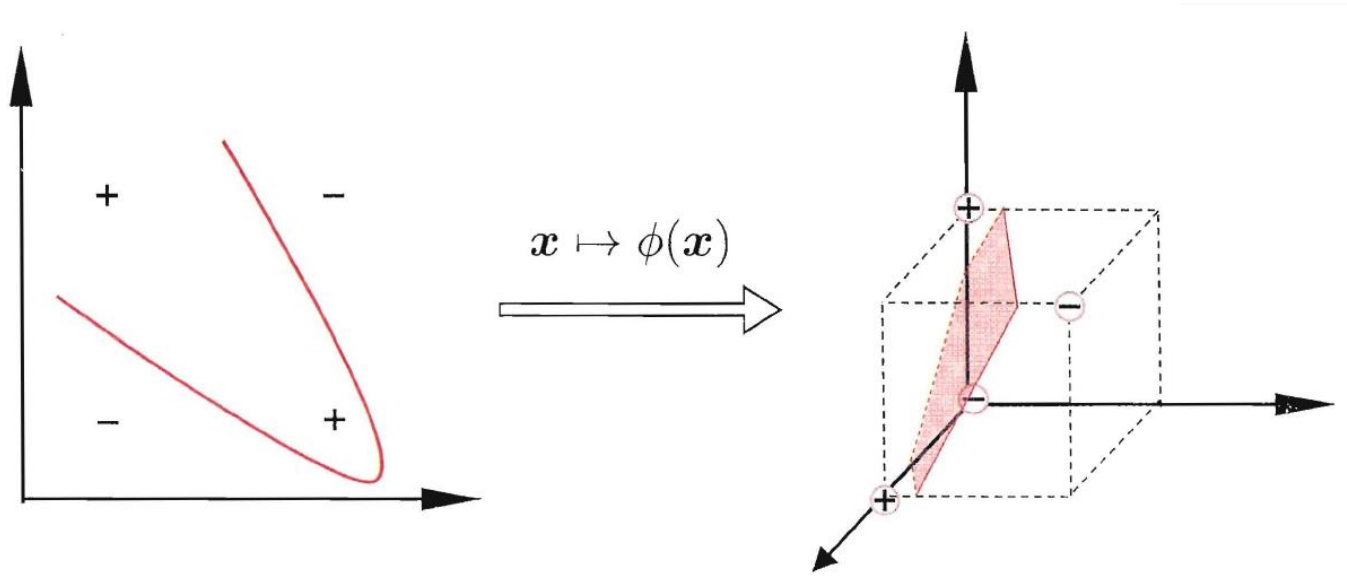


Nonlinear SVM

- What if training data not linearly separable?
  - Naïve: find non-linear decision surface
    - Not efficient
  - Feature transformation: map the original attributes into new feature space and apply linear model in the new space.

Feature Mapping

- Let φ denote the feature mapping
  - $\varphi: \mathbb{R}^m \to \mathcal{H}, \mathbf{x} \mapsto \varphi(\mathbf{x})$
  - where $\mathbb{R}^m$ is the input space and $\mathcal{H}$ is the mapped feature space



Feature Mapping

- Example:
  - $\mathbf{x} = (x_1, x_2)^T \rightarrow \varphi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$
  - $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n) \rightarrow (\varphi(\mathbf{x}_1), y_1), \ldots, (\varphi(\mathbf{x}_n), y_n)$
  - $h(\mathbf{x}) = sign(\mathbf{w}^T\mathbf{x} + b) \rightarrow sign(\mathbf{w}^T\varphi(\mathbf{x}) + b)$
- Apply Linear SVM on mapped feature space, the optimization problem is :

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$$

$$s.t.\ y_i(\mathbf{w}^T\varphi(\mathbf{x}_i)+b) \geq 1, \qquad i=1,2,\ldots,m$$

Feature Mapping

- The dual problem can be written entirely in terms of the inner products of input features ($\mathbf{x}_i^T \mathbf{x}_j$), this means that we would replace all those inner products with $\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$.
- Dual problem:

$$\max_{\alpha} \ \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \, \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$$
$$s.t. \quad \alpha_i \geq 0, \qquad i = 1,2,\dots,m$$
$$\sum_{i=1}^{m} \alpha_i y_i = 0$$

- After solving the dual problem:

$$h(\mathbf{x}) = sign(\mathbf{w}^T \varphi(\mathbf{x}) + b)$$
$$= sign(\sum_{i=1}^{m} \alpha_i y_i \, \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}) + b)$$

Feature Mapping

- Problem:
  - $\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$: inner product of mapped feature vectors, may be expensive to calculate as $\varphi(\mathbf{x}_i)$ may be high dimensional vectors.
  - For efficient calculation, define kernel function.
- Example:
  - Given input instance with feature dimension 2 and the following feature mapping function
    - $\mathbf{x} = (x_1, x_2)^T \rightarrow \varphi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$
    - $\varphi(\mathbf{x})^T \varphi(\mathbf{y}) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)\left(y_1^2, \sqrt{2}y_1y_2, y_2^2\right)^T$
      $= x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (\mathbf{x}^T\mathbf{y})^2$ (shortcut)
  - Inner product can be computed in $\mathbb{R}^2$ (without going to $\mathcal{H}$).

19

Kernel Functions

- Kernel: Given a feature mapping φ, we define the corresponding Kernel to be:

$$\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$$

  - we can get SVMs to learn in the high dimensional feature space given by φ, but without ever having to explicitly find or represent vectors $\varphi(x)$.
- Then, everywhere we have inner product of $\mathbf{x}, \mathbf{y}$ in our algorithm, we could replace it with $\kappa(\mathbf{x}, \mathbf{y})$. Our algorithm would be learning using the features φ.

Kernel Functions

- Dual problem can be re-write as:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \, \kappa(\mathbf{x}_i, \mathbf{x}_j)$$
$$s.t. \quad \alpha_i \geq 0, \qquad i = 1,2, \dots, m$$
$$\sum_{i=1}^{m} \alpha_i y_i = 0$$

- Solving the dual problem and the classifier can be represented:

$$h(\mathbf{x}) = sign(\mathbf{w}^T \varphi(\mathbf{x}) + b)$$
$$= sign(\sum_{i=1}^{m} \alpha_i y_i \, \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}) + b)$$
$$= sign(\sum_{i=1}^{m} \alpha_i y_i \, \kappa(\mathbf{x}_i, \mathbf{x}) + b)$$

Kernel Functions - Examples

- Example:
  - Consider Kernel function: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y})^2$ $(\mathbf{x}, \mathbf{y} \in \mathbb{R}^n)$
  - $\kappa(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} x_i y_i\right)\left(\sum_{i=1}^{n} x_i y_i\right) =$
    $\sum_{i=1}^{n}\sum_{j=1}^{n} x_i x_j y_i y_j = \sum_{i,j=1}^{n}\left(x_i x_j\right)\left(y_i y_j\right) = \varphi(\mathbf{x})^T\varphi(\mathbf{y})$
  - $\varphi(\mathbf{x}) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ \dots \\ x_nx_n \end{bmatrix}, \varphi(\mathbf{y}) = \begin{bmatrix} y_1y_1 \\ y_1y_2 \\ \dots \\ y_ny_n \end{bmatrix}$
  - Calculate $\varphi(\mathbf{x})^T\varphi(\mathbf{y})$ requires $O(n^2)$, whereas calculate $\kappa(\mathbf{x}, \mathbf{y})$ requires $O(n)$.

Kernel Functions - Examples

- Consider another kernel function: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^2$ $(\mathbf{x}, \mathbf{y} \in \mathbb{R}^n)$
  - $\kappa(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} x_i y_i + c\right)\left(\sum_{i=1}^{n} x_i y_i + c\right)$

    $= \sum_{i=1}^{n}\sum_{j=1}^{n} x_i x_j y_i y_j + 2c\sum_{i=1}^{n} x_i y_i + c^2$

    $= \sum_{i,j=1}^{n}(x_i x_j)(y_i y_j) + \sum_{i=1}^{n}(\sqrt{2c}x_i)(\sqrt{2c}y_i) + c^2$
- Corresponding to the feature mapping:

$$\varphi(\mathbf{x}) = (x_1x_1, x_1x_2, \dots, x_nx_n, \sqrt{2c}x_1, \sqrt{2c}x_2, \dots, \sqrt{2c}x_n, c)^T$$

Kernel Functions - Examples

- More broadly, the kernel $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^d$ corresponds to a feature mapping to an $O(n^d)$ feature space (all terms are up to order d).
- However, despite working in this $O(n^d)$-dimensional space, computing $\kappa(\mathbf{x}, \mathbf{y})$ still takes only $O(n)$ time, and hence we never need to explicitly represent feature vectors in this very high dimensional feature space.

Kernel Trick

- Now, lets talk about a slightly different view of kernels.
  - Intuitively, if $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$ are close together, then we might expect $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ to be large.
  - Conversely, if $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$ are far apart, say nearly orthogonal to each other, then $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ will be small.
- So, we can think of $\kappa(\mathbf{x}, \mathbf{y})$ as some measurement of how similar are $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$, or of how similar are $\mathbf{x}$ and $\mathbf{y}$.

Kernel Trick

- Given this intuition, suppose that for some learning problem that you're working on, you've come up with some function $\kappa(\mathbf{x}, \mathbf{y})$ that you think might be a reasonable measure of how similar $\mathbf{x}$ and $\mathbf{y}$ are.
- For instance, choose:

$$\kappa(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}\right)$$

  - This is a reasonable measure of $\mathbf{x}$ and $\mathbf{y}$'s similarity, and is close to 1 when $\mathbf{x}$ and $\mathbf{y}$ are close, and near 0 when they are far apart.
- Gaussian kernel, corresponds to an infinite dimensional feature mapping φ.

Kernel Trick

- But more broadly, given some function κ, how can we tell if it's a valid kernel
  - Can we tell if there is some feature mapping φ so that $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ for all $\mathbf{x}, \mathbf{y}$?
- Theorem (Mercer):
  - Let κ: $\mathbb{R}^n \times \mathbb{R}^n \longmapsto \mathbb{R}$ be given. Then for κ to be a valid kernel, it is necessary and sufficient that for finite number of points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, the corresponding kernel matrix is symmetric positive semi-definite.
  - Kernel matrix for m points are $m \times m$ matrix K, and the (i, j)-entry is $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$.
- In real application, usually use the existing kernel functions.

Kernel Functions

- Common kernels:
  - Linear: $\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T\mathbf{y}$
  - Polynomial: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^d$
    - d is the degree of the polynomial
  - Gaussian: $\kappa(\mathbf{x}, \mathbf{y}) = \exp(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2})$ (or $\kappa(\mathbf{x}, \mathbf{y}) = \exp(-\gamma\|\mathbf{x} - \mathbf{y}\|^2)$)
    - Corresponds to an infinite dimensional feature space
    - Usually the first choice.
    - radial basis function (RBF)
    - https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

Kernel Functions

- Kernel: substitutes the inner product and act as a nonlinear similarity measure
  - Kernel implicitly defines the feature space
  - Any algorithm that depends only on inner products can use the kernel trick!
  - Perform feature scaling before applying kernel trick.

How to Choose Kernel Function

- Select kernel function based on expert or prior knowledge of the data; if no prior knowledge, try cross validation.
  - Grid search for hyper-parameter/parameter setting.
  - https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf
- Suppose n is the number of features, m is the number of samples
  - If n is large (compare to m): linear kernel.
  - If n is small, m is intermediate: RBF kernel
  - If n is small, m is large: create/add more features and use linear model.

Chapter 3.4: Emsemble Learning

Ensemble Learning

- An ensemble learning system is obtained by combining diverse models (henceforth classifiers).
  - Also known as multi-classifier system, committee-based learning or just ensemble systems.
- We use such approach routinely in our daily life by asking the opinions of several experts before making a decision:
  - Ask the opinions of several doctors before agreeing to a medical procedure;
  - Read user reviews before purchasing an item (particularly big ticket items);
  - Evaluate future employees by checking their references.

Ensemble Learning

- An ensemble contains a number of learners which are usually called base learners. The generalization ability of an ensemble is usually much stronger than that of base learners.
- Actually, ensemble learning is appealing because that it is able to boost weak learners which are slightly better than random guess to strong learners which can make very accurate predictions.
  - Base learners are also referred as weak learners.

Ensemble Learning

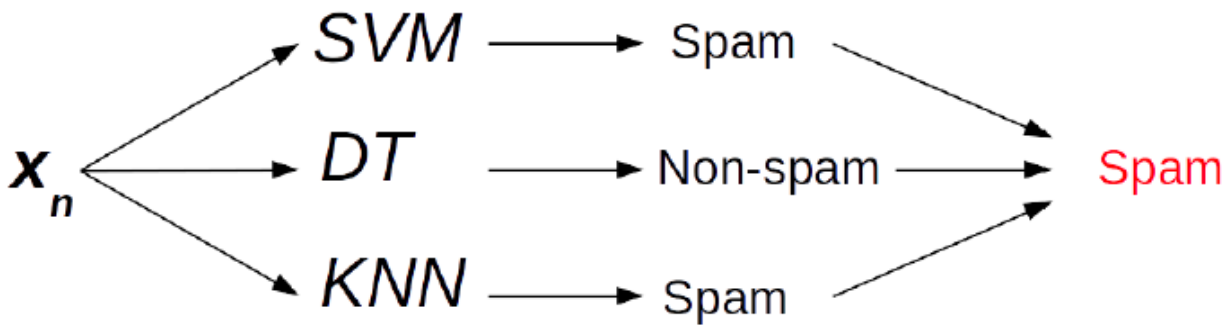
- Base learners are usually generated from training data by a base learning algorithm which can be
  - decision tree
  - neural network
  - or other kinds of machine learning algorithms.
- Most ensemble methods use a single base learning algorithm to produce homogeneous base learners.
- Also some methods which use multiple learning algorithms to produce heterogeneous learners.
  - No single base learning algorithm and thus, some people prefer calling the learners individual learners or component learners.

Ensemble Construction

- Typically, an ensemble is constructed in two steps.
  - First, a number of base learners are produced, which can be generated in a parallel style or in a sequential style where the generation of a base learner has influence on the generation of subsequent learners.
  - Then, the base learners are combined to use, where among the most popular combination schemes are majority voting for classification and weighted averaging for regression.

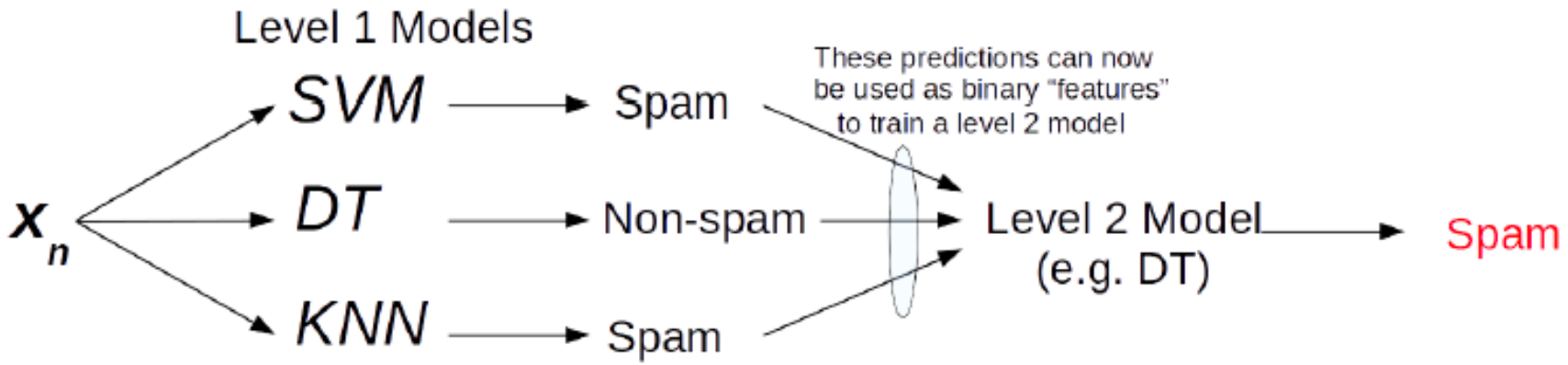
Ensembles Strategies

- Voting or averaging of prediction of multiple pre-trained models.



Ensembles Strategies

- Stacking: use predictions of multiple models as “features” to train a new model and use the new model to make predictions on test data.



Ensemble: Another Strategy

- Instead of training different models on same data, train same model multiple times on different data sets , and "combine" these "different" models.
  - Use some simple/weak model as the base model/learning algorithm.
  - How to generate different data sets? (In practices, we only have one data set at training time.)

Ensemble Methods

- There are many effective ensemble methods. We will briefly introduce two representative methods:
  - Bagging
  - Boosting
- We will consider binary classification for simplicity.
  - Training set: $D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$
  - Input feature: $x^{(1)} \in \mathbb{R}^m$, label: $y^{(i)} \in \{+1, -1\}$.

Bagging

- Bagging stands for Bootstrap Aggregation
  - trains a number of base learners each from a different bootstrap sample by calling a base learning algorithm.
- A bootstrap sample is obtained by subsampling the training data set with replacement, where the size of a sample is as the same as that of the training data set.
  - For a bootstrap sample, some training examples may appear but some may not, where the probability that an example appears at least once is about 0.632.
  - Probability that an instance does not show up in sample is

$$\lim_{m\to\infty}(1-\frac{1}{m})^m=\frac{1}{e}\approx 0.368.$$

Bagging

- Given original data set D with m training samples, create T bootsrap samples $\{D_t\}_{t=1}^{T}$
  - Each sample D_t is generated from D by sampling with replacement
  - Each sample D_t has the same number of examples as in data set D
  - These samples are reasonably different from each other
- Train base learners $h_1, h_2, \ldots, h_T$ using $D_1, D_2, \ldots, D_T$
- Combine the results by majority voting

$$H(x) = \underset{y \in \{+1,-1\}}{\operatorname{argmax}} \sum_{t=1}^{T} \mathbb{I}(y = h_t(x))$$

Bagging Algorithm

Input: $D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

Base learning algorithm $\mathcal{L}$

Number of learning rounds T

Process:

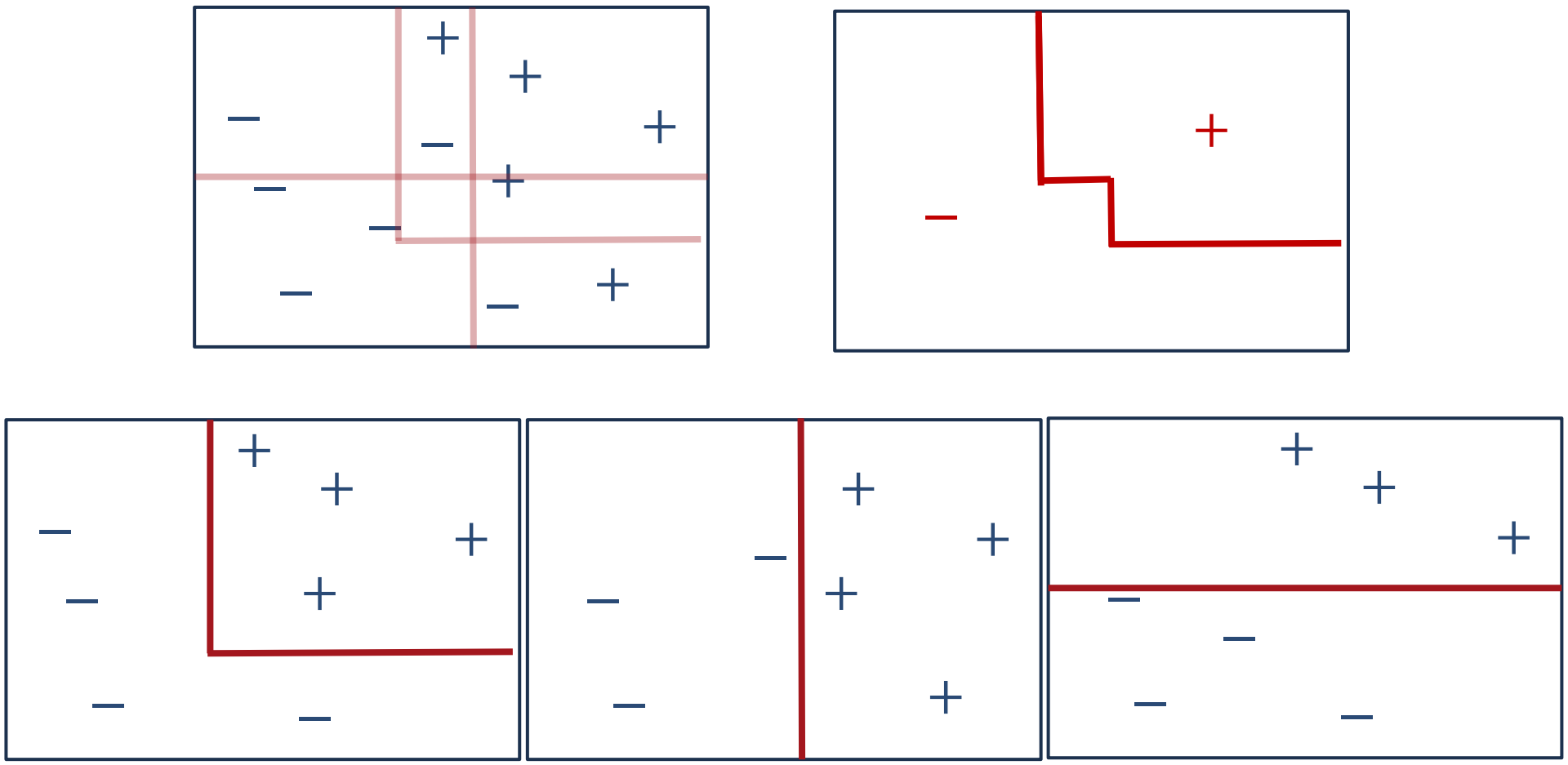
for $t = 1,2, \dots, T$:

$D_t = Bootstrap(D)$

$h_t = \mathcal{L}(D_t)$

Output: $H(x) = \underset{y \in \{+1,-1\}}{\operatorname{argmax}} \sum_{t=1}^{T} \mathbb{I}(y = h_t(x))$

Bagging Illustration

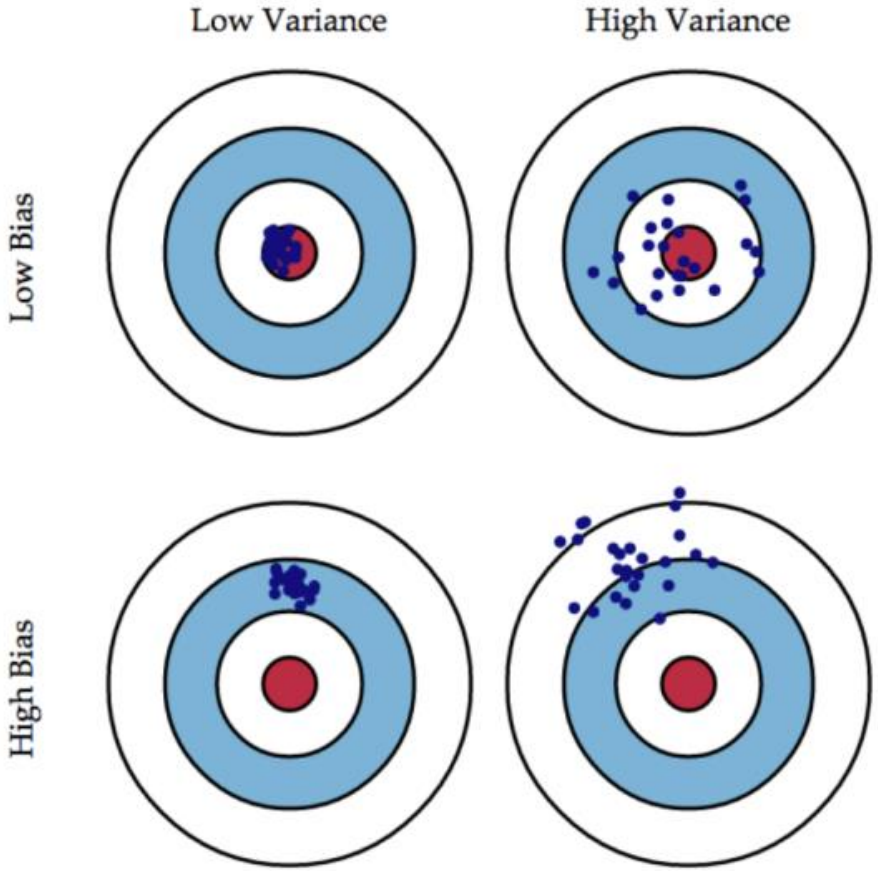


43

Bias and Variance

Let $f(x)$ be the function our data follows and $\hat{f}(x)$ is our model

- $bias$: $E\left[\hat{f}(x)\right] - f(x)$
  - difference between its expected prediction value and the true value
- $variance$: $E[(\hat{f}(x) - E[\hat{f}(x)])^2]$
  - measure of the dispersion of prediction model
  - Sensitivity to different data samples



Averaging Reduce Variance

- Assume we measure a random variable x with a $N(\mu, \sigma^2)$ distribution
  - If only one measurement x_1 is done
    - Mean of the measurement is $E(x_1) = \mu$
    - Variance $Var(x_1) = \sigma^2$
  - If random variable is measured K times $(x_1, x_2, .., x_K)$, the value is estimated as $\frac{x_1+x_2+\cdots+x_K}{K}$
    - Mean of the estimation is $E\left(\frac{x_1+x_2+\cdots+x_K}{K}\right) = E(x_i) = \mu$
    - Variance $\text{Var}\left(\frac{x_1+x_2+\cdots+x_K}{K}\right) = \frac{1}{K} Var(x_i) = \frac{\sigma^2}{K}$

When Bagging Helps?

- Observe: bagging is a kind of average
- Main property of bagging:
  - Bagging decreases variance of the base model without changing the bias.
- Bagging typically helps
  - When applied with an over-fitted base model. High dependency on actual training data.
- Bagging does not help much
  - High bias. When the base model is robust to the changes in the training data.

Random Forest

- An ensemble of decision tree (DT) classifiers
- Use bagging on data samples
- For each bootstrap sample, derive a decision tree
  - In each node, uses bagging on features
  - Given a total of d features, each node uses $\log |d|$ randomly chosen features.
  - Split the node using splitting criterion
- Each node will split the training data differently at leaves
- Prediction for a test example votes on/averages predictions form all DTs.

Boosting

- Basic idea:
  - Take a weak learning algorithm (slightly better than random)
  - Turn it into an awesome one by making it focus on difficult samples
- Most boosting algorithms follows these steps:
  1. Train a weak model on some training data
  2. Compute the error of the model on each training sample
  3. Give higher importance to examples on which the model made mistakes
  4. Re-train the model using "importance weighted" training examples
  5. Go back to step2.
- Boosting is in fact a family of algorithms since there are many variants. Here, the most famous algorithm, AdaBoost is considered as an example.

AdaBoost

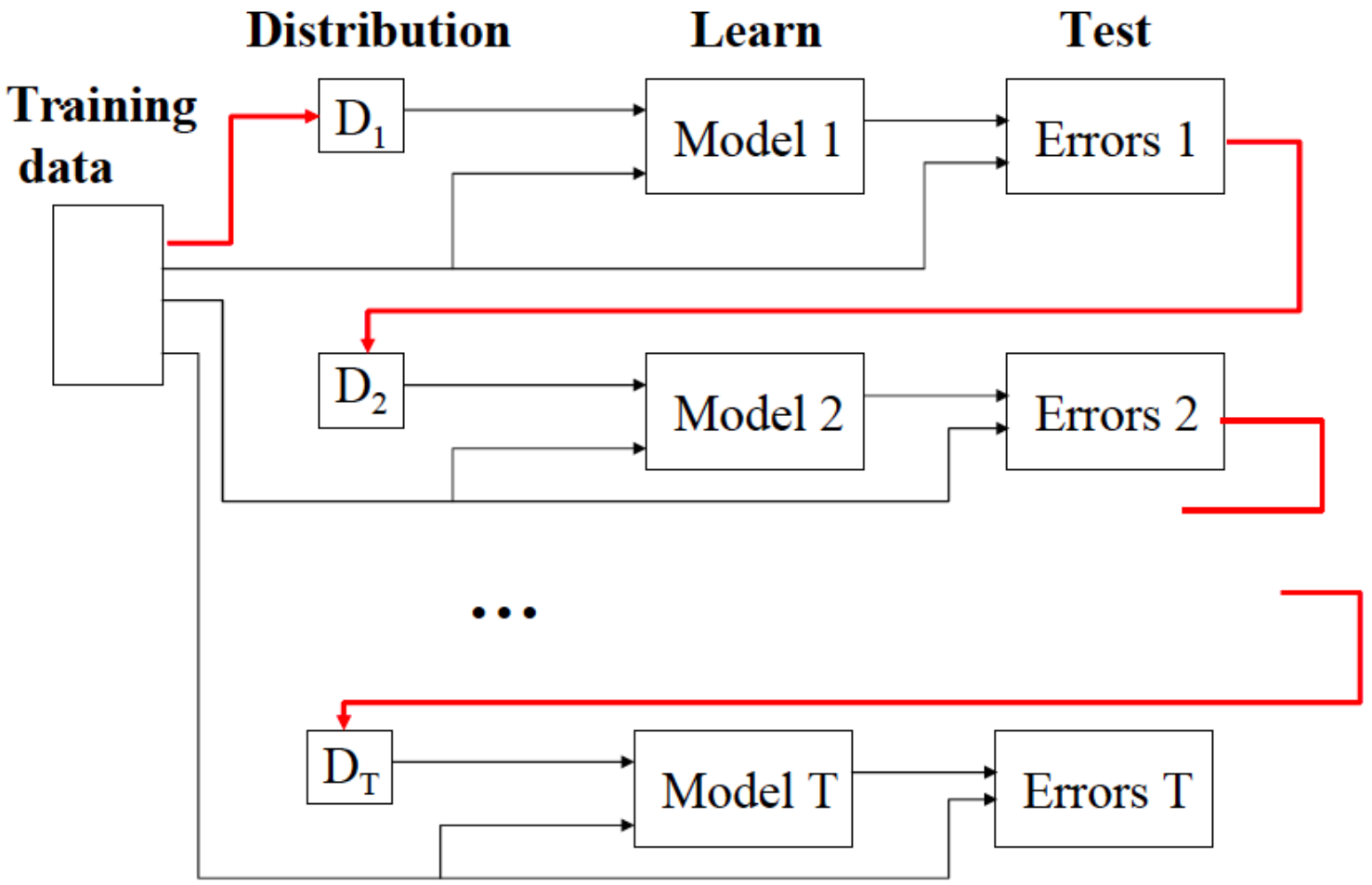
- Denote the distribution of the weights at the t^{th} learning round as D_t

1. Initialize equal weight to all the training example, $D_1(n) = 1/n$.
2. From the training data set and D_t, the algorithm generates a base learner h_t by calling the base learning algorithm.
3. Then, it uses the training examples to test h_t, and the weights of the incorrectly classified examples will be increased.
   - Update weight distribution D_{t+1} is generated.

AdaBoost

- Repeat such procedure for T times, each of which is called a round
  - final learner is derived by weighted majority voting of the T base learners, where the weights of the learners are determined during the training process
  - $H(x) = sign(\sum_{t=1}^{T} \alpha_t h_t(x))$
- In practice, the base learning algorithm may be a learning algorithm which can use weighted training examples directly; otherwise the weights can be exploited by sampling the training examples according to the weight distribution D_t.

AdaBoost


Distribution
Learn
Test
Training data
D_1
Model 1
Errors 1
D_2
Model 2
Errors 2
...
D_T
Model T
Errors T

AdaBoost Algorithm

Input: $D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

Base learning algorithm $\mathcal{L}$

Number of learning rounds T

Process:

$D_1(x) = 1/\mathrm{m}$

for $t = 1,2, \dots, T$:

$h_t = \mathcal{L}(D, D_t)$

$\epsilon_t = \sum_{i=1}^{m} D_t(i)\, \mathbb{I}\left(y^{(i)} \neq h_t\left(x^{(i)}\right)\right)$ # compute weighted errors

$\alpha_t = \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ # set importance of h_t

$$D_{t+1}(x) = \frac{D_t(x)}{Z_t} \cdot \begin{cases} \exp(-\alpha_t), if\ \ h_t(x) = y \\ \exp(\alpha_t), if\ \ h_t(x) \neq y \end{cases}$$

update distribution

Z_t is normalization factor which enables D_{t+1} to be a distribution

Output: $H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$

AdaBoost Examples

- Consider binary classification with 10 training examples

ID	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- Each of our weak classifiers will be an x-axis parallel linear classier (decision stump or one level decision tree).
- Initial setting of weight distribution:

$$D_1 = [0.1, 0.1, \ldots, 0.1]$$

AdaBoost Examples

ID	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- **Round 1**: classifier $h_1(x) = \begin{cases} 1, & if\ x < 2.5 \\ -1, & if\ x > 2.5 \end{cases}$
  - Error:$\epsilon_1 = 0.1 * 3 = 0.3$
  - Importance: $\alpha_1 = \frac{1}{2}\log(\frac{1-\epsilon_1}{\epsilon_1})$=0.4236
  - Update weight distribution:
  - $D_2(i) = \frac{0.1}{z} * \exp(-0.4236) = 0.0715 \quad for\ i = 1,2,3,4,5,6,10$
  - $D_2(i) = \frac{0.1}{z} * \exp(0.4236) = 0.1666 \quad for\ i = 7,8,9$

AdaBoost Examples

ID	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- **Round 2**:
- $D_2 = [0.0715,0.0715,0.0715,0.0715,0.0715,0.0715,0.1666,0.1666,0.1666,0.0715]$
- classifier $h_2(x) = \begin{cases} 1, & if\ \ x < 8.5 \\ -1, & if\ x > 8.5 \end{cases}$
  - Error:$\epsilon_2 = 0.0715 * 3 = 0.2145$
  - Importance: $\alpha_2 = \frac{1}{2}\log(\frac{1-\epsilon_2}{\epsilon_2})$=0.649
  - Update weight distribution:
  - $D_3 = [\ 0.045,0.045,0.045,0.167,0.167,0.167,0.106,0.106,0.106,0.045]$

AdaBoost Examples

ID	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- **Round 3**:
  - $D_3 = [\ 0.045,0.045,0.045,0.167,0.167,0.167,0.106,0.106,0.106,0.045]$
- classifier $h_3(x) = \begin{cases} -1, & if\ \ x < 5.5 \\ 1, & if\ x > 5.5 \end{cases}$
  - Error:$\epsilon_3 = 0.045 * 4 = 0.18$
  - Importance: $\alpha_3 = \frac{1}{2}\log(\frac{1-\epsilon_3}{\epsilon_3})$=0.758
  - Update weight distribution:
  - $D_4 = [\ 0.125,0.125,0.125,0.102,0.102,0.102,0.065,0.\ 065,0.065,0.125]$

AdaBoost Examples

ID	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1
$H(x)$	1	1	1	-1	-1	-1	1	1	1	-1

- Output classifier after three rounds:
  - $h_1(x) = \begin{cases} 1, & if\ x < 2.5 \\ -1, & if\ x > 2.5' \end{cases}, h_2(x) = \begin{cases} 1, & if\ x < 8.5 \\ -1, & if\ x > 8.5' \end{cases}$
  - $h_3(x) = \begin{cases} -1, & if\ x < 5.5 \\ 1, & if\ x > 5.5 \end{cases}$
  - $H(x) = sign(0.4236 * h_1(x) + 0.649 * h_2(x) + 0.758 * h_3(x))$
  - Error: 0.
- Multiple weak, linear classifiers combined to give a strong, nonlinear classier.

Boosting

- Boosting algorithms can be shown to be minimizing an exponential loss function:

$$l_{exp}(H|D) = \sum_{i=1}^{m} \exp\{-y^{(i)} H(x^{(i)})\}$$

  - where $H(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
  - For proof, refer to 14.3 boosting in “Pattern Recognition and Machine Learning”.
- Boosting will sequentially minimize the loss function, will reduce the bias.

Bagging vs Boosting

- No clear winner, usually depends on the data

1. Bagging is computationally more efficient than boosting
   - Bagging can train the M models in parallel, boosting cannot.
2. Both reduce variance (and overfitting) by combining different models.
   - The resulting model has higher stability as compared to individual ones.
3. Bagging usually cannot reduce the bias, boosting can.
4. Bagging usually performs better than boosting if we do not have a high bias and only want to reduce variance
   - Higher correlation between models.